Shiksha Setu: A Measurement-Driven, Local-Retrieval AI Tutoring Platform for Multilingual Indian Education

Rachit Ranka, Aaditya Saini, Priyanka Devi, Yogesh Kohli, Ruhani Mehta *Department of Computer Science Engineering*
Chandigarh University, Mohali 140413, Punjab, India
rankarachit5@gmail.com, aadityasaini899@gmail.com, pjangra9105@gmail.com,
ykohli2005@gmail.com, ruhanimehta019@gmail.com

***Abstract*—Existing AI tutoring systems depend on cloud infrastructure, operate primarily in English, and are not aligned to a national curriculum. This paper presents Shiksha Setu, a local-retrieval tutoring platform for the Indian NCERT curriculum in which the corpus, its embeddings and the whole of retrieval are local. Generation is a hosted call, and the paper is explicit about which half is which, and does not claim to be offline. All results are measured against 153 ingested textbooks spanning classes 1–12 (1,542 chapters, 42,995 indexed passages), a corpus state recorded so the numbers are interpretable. Retrieval is evaluated by a five-configuration ablation over 22 queries in eleven Indian languages: a lexical baseline retrieves *nothing at all*, because a Devanagari or Perso-Arabic query shares no tokens with an English index. Dense retrieval alone reaches 11 of 22, query rewriting alone 14, and pooling both, the deployed configuration, at 16 ± 1. A grounding score separating answers written from their retrieved passages from answers that drifted is calibrated on 256 pairs, not four examples, moving the shipped threshold from 0.55, which admitted 35 of 240 negatives, to 0.715, which admits none. The 0.360 separation reproduces under character n-gram TF-IDF and a second encoder family, so it is a property of the answers and not of one embedding space. Half-precision inference halves the embedding footprint to 1,083 MB at a mean cosine fidelity of 0.999998, verified by kernel-level peak resident set, enabling deployment on 4 GB machines. The pipeline audits its own sources, detecting legacy Devanagari fonts that render 66% of Hindi textbooks as Latin gibberish, and a security audit repairs twelve vulnerabilities including a safety pipeline that failed open.**

***Index Terms*—Retrieval-Augmented Generation, Multilingual Education, Local-Retrieval AI, Indian Languages, Curriculum-Aligned Tutoring, Accessibility, NCERT**

I. Introduction

India is home to 1.4 billion people speaking 22 constitutionally recognized languages [30], written in more than a dozen scripts. The National Council of Educational Research and Training (NCERT) publishes the curriculum that governs instruction for the 24.7 crore children enrolled in school [29], yet its textbooks exist only in English, Hindi, and Urdu. A Tamil-speaking student in rural Tamil Nadu, a Marathi-speaking student in Maharashtra, and a Bengali-speaking student in West Bengal all study the same curriculum, but none of them can read it in their mother tongue unless a human teacher translates it in real time.

Artificial intelligence can bridge this gap, but the dominant paradigm of cloud-hosted large language models introduces three structural failures for Indian education:

Language exclusion. Most educational AI systems operate in English. English is a first language for a negligible share of the population and a second or third language for roughly one Indian in ten [30], so for the great majority such systems are inaccessible at the point of need. **Infrastructure dependence.** Cloud solutions need consistent bandwidth and bill per query, which makes sustained use impractical in the towns and villages where most students are. **Pedagogical misalignment.** A general-purpose model has no notion of curriculum structure or grade-appropriate complexity, and a class 6 student asking about chemical reactions needs a different answer from a class 12 student on the same topic.

This paper presents Shiksha Setu, a platform that addresses the first and third failures and *partially* the second, through a measurement-driven engineering approach.

The qualification is deliberate. Retrieval is entirely local. The corpus, its embeddings and the vector index never leave the machine, but generation is served by a hosted endpoint, so a question's text does leave it and does incur a per-query cost. The system therefore removes the dependency for the component that dominates storage and latency without removing it altogether. Local generation is configured and available (Section III), but a language model cannot share a 4 GB machine with the embedder, so the offline configuration and the 4 GB configuration are alternatives, not the same system. We say local-retrieval rather than local-first because the latter is read as "offline", and this system answers nothing without a network. The measurements in this paper were taken in the hosted configuration. Every claim is backed by an experiment whose procedure, data, and result are reported, including experiments that produced wrong results before being corrected. The contributions are:

This work contributes an ingestion pipeline covering all 559 NCERT textbooks, which detects and rejects the legacy-font corruption that renders 27 of 41 Hindi books unreadable, a defect found only by measuring for it. It contributes a cross-lingual retrieval system that answers questions in eleven Indian languages from an English corpus, with a controlled ablation over 22 queries establishing that query rewriting and

direct embedding fail on disjoint inputs, so that pooling them retrieves correctly for 16 ± 1 of 22 where either alone manages 11 or 14 and a lexical baseline manages none. It contributes a memory strategy verified by kernel-level measurement, which brings the serving pipeline inside a 4 GB budget and corrects an earlier arithmetic estimate that understated consumption by 31%. And it contributes a grounding metric that detects when fluent, confident answers have drifted from their source material, a failure indistinguishable from a correct answer without quantitative measurement, calibrated here on 256 answer and passage pairs instead of hand-checked examples and compared against three alternative estimators. The journal version additionally reports akshara-level readability measurement for nine Brahmic scripts, which replaces Latin-alphabet syllable counting with script-appropriate segmentation, and a security audit documenting twelve vulnerabilities, all repaired, among them a safety pipeline that failed open because of an import error.

II. Related Work

Lewis et al. [2] introduced retrieval-augmented generation (RAG) for knowledge-intensive NLP tasks, since surveyed at length [27], establishing the paradigm of conditioning language model output on retrieved passages. Shiksha Setu adopts this architecture with a critical modification: a grade-level filter inserted between retrieval and generation, because the same topic at class 6 and class 12 requires different explanations. Karpukhin et al. [3] demonstrated the superiority of dense passage retrieval over BM25. Ma et al. [4] showed that query rewriting before embedding improves retrieval quality. The technique that resolves the romanized Hindi failure of Section VI. Nogueira and Cho [5] established passage re-ranking with cross-encoder models. The BGE-Reranker component exists in the codebase but is not yet wired into the retrieval path.

Chen et al. [6] presented BGE-M3, a multilingual embedding model supporting over 100 languages in a unified 1024-dimensional vector space. Its cross-lingual property, that semantically equivalent passages in different languages map to nearby vectors, allows a Hindi question to retrieve an English passage without explicit translation. Gala et al. [7] developed IndicTrans2 covering all 22 scheduled Indian languages, serving as the translation backbone. Kakwani et al. [1] provided monolingual corpora and evaluation benchmarks for Indian languages. Khanuja et al. [8] trained MuRIL on transliterated text, directly relevant to the romanized Hindi failure documented in Section VI.

Recent literature categorizes multilingual retrieval into frameworks such as MultiRAG (direct retrieval) and tRAG (translation-RAG). The platform described in this work formally implements a tRAG architecture [9], rewriting queries to a high-resource language (English) before retrieval. This architectural choice unknowingly resolves a newly documented vulnerability in large language models: Output Language Drift [10]. Studies show that when a multilingual RAG system processes a query in a target language but retrieves context in a dominant high-resource language, the model mixes languages and outputs the final answer in the high-resource language. The strict separation of the generation language from the delivery language via the translation layer elegantly neutralizes this cognitive drift.

The retrieval layer uses the HNSW graph structure proposed by Malkov and Yashunin [11], implemented through the pgvector extension for PostgreSQL. The index parameters ($m = 16$, $ef\_construction = 64$) balance build time against recall. The resulting search latency is reported in Fig. 1, not here. Reimers and Gurevych [12] established the sentence-level embedding formulation used to load BGE-M3.

Zorzi et al. [13] found that extra-large letter spacing improved reading speed and halved errors in dyslexic children, the single typographic intervention with strong experimental support and the default accessibility control in the platform. Rello and Baeza-Yates [14] and Kuster et al. [15] found no reliable advantage for dyslexia-specific typefaces. Nag [16] and Nag and Snowling [17] demonstrated that akshara-based literacy develops differently from alphabetic literacy, motivating the akshara-level segmentation described in Section VIII.

Kincaid et al. [18] derived the Flesch-Kincaid grade-level formula defined on English syllable counts, which is why the platform refuses to compute it for Devanagari text. Xu et al. [19] demonstrated that generic simplification routinely loses content, motivating the measure-and-keep-the-easier approach. Micikevicius et al. [20] established the safety of half-precision arithmetic for neural network inference, providing the theoretical basis for the fp16 optimization that halves the BGE-M3 memory footprint.

Table I compares this work with the systems an Indian student is most likely to encounter: Khanmigo [21], Khan Academy's GPT-4 tutor; BYJU'S, the largest Indian ed-tech platform; and Bhashini [22], the Government of India's national language-technology mission.

The comparison is confined to *architectural properties*. Every cell is a structural fact, a figure the vendor has published, or *n/d* where they have not, and no row sets a measurement of ours against a number we could not obtain. Commercial products publish neither latency distributions nor retrieval quality nor memory footprints, and a table of plausible estimates for them would be a table of invented numbers set in the typography of measurement.

TABLE I
Comparison with deployed systems. *n/d*: not disclosed by the operator. Rows are architectural properties, not performance claims.

Property	Khanmigo	BYJU'S	Bhashini	Shiksha Setu
NCERT-aligned corpus	no	yes	—	yes
Indian languages	no	n/d	22	11
Corpus held locally	no	no	—	yes
Retrieval runs locally	no	no	—	yes
Generation runs locally	no	no	—	no
Runs without a network	no	no	no	no
Brahmic readability aid	no	no	no	yes
Per-answer grounding	n/d	n/d	—	yes
Corpus corruption audit	n/d	n/d	—	yes
Stated memory target	n/d	n/d	n/d	4 GB
Tutoring function	yes	yes	no	yes

Bhashini is dashed on several rows, not marked absent, because it is language infrastructure (translation, speech, translit-

eration) and not a tutor, so corpus and retrieval questions do not apply. It is included because its 22 languages [22] exceed ours, and a platform claiming multilingual reach should be measured against the strongest such system.

Three rows are stated against our own interest: generation is not local, the system needs a network, and BYJU'S covers our curriculum at a scale we do not approach. What distinguishes this work is the rest. The corpus, index and retrieval are local, a grounding score accompanies every answer, and the pipeline audits its own sources instead of embedding whatever it extracts. We have found no comparable system reporting the last two, which is a claim about the published record and not about what those systems do internally.

III. System Architecture

A student's question traverses an eight-stage pipeline from input to response, six of them mandatory. The latencies below were taken on the development machine; what the platform *requires* is the 2,094 MB of Section VI, which is what puts it inside 4 GB. Fig. 1 shows those six with their measured latencies on an 8 GB Apple M1, arranged by the property this paper is about, which is which side of the machine boundary each one runs on.

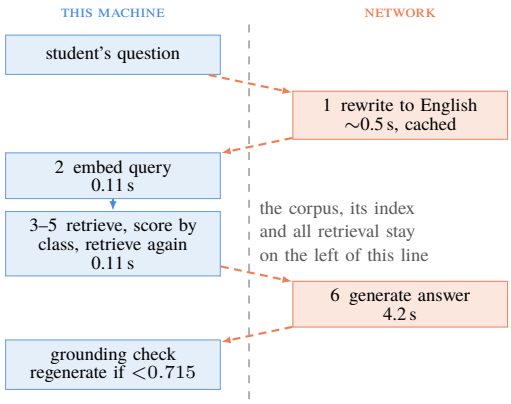

Fig. 1. The mandatory request path. Medians over 15 warm requests, five questions run three times on an 8 GB Apple M1. The dashed line is the machine boundary: the corpus and all retrieval stay left of it. Generation dominates and is least predictable, 4.2 s median against a 95th percentile of 13.2 s, because it is hosted. All of retrieval is 0.11 s. The grounding check carries no latency because it was not measured separately.

The design separates the question's language from the answer's language. A student may type in romanized Hindi and receive a response in Tamil. This independence is critical because NCERT publishes in only three languages while the platform serves eleven. Translation is a serving-layer responsibility handled by IndicTrans2 [7].

The platform integrates twenty-eight model identifiers, of which **five** are exercised on the critical path and measured in this work. The remainder are configured alternatives that this evaluation does not characterize. Table II separates the two, because a model stack presented as a single list would imply an empirical basis that only the first group has.

Exactly one model is resident locally on the critical path. This is not an accident of packaging but the property that makes the memory budget of Section VI achievable. Embedding must be local because every chunk in the corpus passes through it during ingestion, whereas generation, illustration and OCR are per-request and therefore hosted, leaving no language-model weights resident during serving.

TABLE II
Model Stack. Upper block: measured on the critical path. Lower block: integrated, not evaluated here. † denotes a hosted model; all others are resident locally.

Role	Model	Params
Embeddings, grounding	BGE-M3	568M
Generation, rewriting	LLaMA-3.1-8B†	8B
Illustration	FLUX.1-dev†	12B
Vision OCR	Nemotron-Nano-12B†	12B
Generation (compared)	LLaMA-3.3-70B†	70B
Translation	IndicTrans2-1B	1B
Speech-to-Text	Whisper V3 Turbo [23]	809M
Text-to-Speech	MMS-TTS (11 ckpts)	∼100M
Curriculum Valid.	Gemma-2-2B-IT	2B
Text Simplification	Qwen2.5-3B	3B
Reranking	BGE-Reranker-v2-M3	568M

BGE-M3 is used twice for distinct purposes: retrieving passages, and later scoring how far an answer strayed from them (Section V-D). Both uses depend on the same property, cross-lingual embedding in a shared space, which is what permits a Devanagari passage and an English answer to be compared numerically at all.

Two integrated models are named but unusable in this deployment and are reported as such, not omitted. The local OCR path (GOT-OCR2) requires CUDA and cannot execute on Apple silicon. Its Python module additionally failed to import because `torchvision` was an undeclared dependency, which presented as a hardware limitation for some time before being diagnosed as a packaging one. The Tesseract fallback is installed without Devanagari training data, leaving it unable to read the script that most needs recovery (Section IV-C).

The choice of the 8B model over the 70B variant was driven by measurement (Table III).

TABLE III
Text generation model latency: one fixed prompt at approximately 70 completion tokens, against the same hosted endpoint. These are model-selection figures and are shorter than the end-to-end generation stage of Fig. 1, which produces a full answer.

Model	Latency
meta/llama-3.1-8b-instruct	**0.9 s**
nvidia/nemotron-nano-9b-v2	2.5 s
meta/llama-3.3-70b-instruct	17–125 s
nvidia/llama-3.1-nemotron-nano-8b-v1	Timeout

The persistence layer combines PostgreSQL 17 with pgvector for HNSW-indexed similarity search, Redis 7 for response caching, and SQLite for the third cache tier. The multi-tier architecture (L1: in-memory LRU → L2: Redis → L3: SQLite) serves read-heavy endpoints. Cache keys incorporate a SHA-256 hash of the authorization header to prevent cross-user leakage. The vector index uses cosine distance:

```
CREATE INDEX idx_emb_hnsw_cosine
  ON embeddings USING hnsw
  (embedding vector_cosine_ops)
  WITH (m=16, ef_construction=64);
```

An earlier migration stored embeddings as `double precision[]` instead of `vector(1024)`, making index creation impossible and condemning every search to a sequential scan.

All measurements in this paper were taken on an Apple M1 with 8 GB of unified memory. We report no figures for hardware we did not run on.

IV. Corpus Construction

NCERT encodes every textbook with a five-character code: `jesc1` represents Class 10, English medium, Science, book 1. The subject letters are not algorithmically derivable. Science is `sc` for classes 7–10 but `cu` (Curiosity) for class 6, and splits into `ph`, `ch`, `bo` at classes 11–12. The catalog is scraped from NCERT's textbook picker and cached locally.

The complete catalog contains **559 textbooks**: 209 in English, 191 in Hindi, and 159 in Urdu. The platform teaches from a deliberate subset of **263 books**, every English edition, plus the 54 subjects that exist only in Devanagari.

The 159 Urdu editions are excluded from the taught corpus entirely, and are neither ingested nor retrieved from. They are translations of books already held in English, so including them would triple the storage for that curriculum while adding no content, and cross-lingual retrieval was observed to cite an Urdu chapter as the source of an English answer, a citation the student can neither read nor check. Urdu remains available as an *answer* language, and appears as a query language in the retrieval evaluation of Section VI: what a student writes in and reads back is independent of which edition the system learned from.

The pipeline processes each book through five stages: ZIP download, per-chapter PDF extraction via PyMuPDF, text cleanup, semantic chunking (1200 characters, 200-character overlap, paragraph-boundary preference), BGE-M3 embedding (fp16), and storage in PostgreSQL. Each book is committed as a single database transaction for resumability. Download and embedding contend for no shared resource. One waits on a remote web server, the other saturates the GPU. Executed sequentially they idle in turn, so the next book's archive is fetched on a prefetch thread while the current book is embedded.

The speedup was established by a controlled comparison, not inferred. The same three books (classes 6–8 *Curiosity*) were ingested twice, once with prefetching disabled through `INGEST_PREFETCH=0`, producing byte-identical output in both runs (37 chapters, 1,130 chunks): **193 s per book sequentially against 140 s per book with prefetching, a 1.38× speedup**. Extrapolated, the 153 obtainable books require approximately 6.0 hours on a single Apple M1. The figure is quoted over the obtainable books instead of the full 263-book curriculum, because the remainder cannot be ingested at any rate.

We note that a naive comparison of figures taken under different conditions would have reported this backwards. An earlier sequential measurement of 97 s per book was recorded on a different, smaller book set. Read against the 140 s figure it appears to show prefetching making ingestion *slower*. Only the paired measurement on identical inputs is informative, and unpaired throughput figures are excluded here for that reason.

Two-thirds of the Hindi curriculum is rendered unreadable by every PDF text extractor tested. NCERT's older Hindi textbooks are typeset in **Walkman-Chanakya 905**, a legacy font mapping Devanagari glyphs onto ASCII codepoints with no `ToUnicode` table. Text extraction returns raw byte values. The analysis covers 41 of the 54 Devanagari-only books in the taught curriculum. The method samples a single chapter PDF per book, and for the remaining 13 books NCERT publishes no individual chapter files, returning HTTP 404 for chapters 1 through 3 at the documented URL scheme. Those 13 are absent for lack of a retrievable sample, not by selection, and no result here is conditioned on their contents (Table IV):

TABLE IV
Hindi Corpus Corruption Analysis

Extraction Category	Books	Devanagari
Legacy font (Walkman-Chanakya)	**27**	0.0%
Unicode font (Kokila, Arya)	14	97.9–100%

The measurement that was inverted. An earlier quality assessment counted broken Devanagari sequences and reported the 27 legacy-font books as the *cleanest* in the corpus, because a detector searching for damaged Devanagari finds nothing in a book containing no Devanagari at all. 133 chunks of Latin gibberish entered the retrieval corpus before the error was caught. The corrected detection identifies Hindi books whose extracted text is overwhelmingly Latin.

PyMuPDF's text layer is lossy on mathematical content. Class 10 Mathematics chapter 4 defines a quadratic as $ax^2 + bx + c,\ a \neq 0$. The text layer yields "$ax2 + bx + c,\ a\ \ 0$": the superscript flattened and the inequality deleted. Pages requiring OCR are identified by image density (Table V).

TABLE V
Image Density as OCR Predictor

Page Type	Images/1000 chars
Mathematics with equations	30.8
Science with figures	5.9
Prose with occasional figures	2.0–4.0

Cloud OCR (Nemotron VL) correctly recovers symbols but introduces substitution errors worse for learners. The text layer loses glyphs. The vision model substitutes plausible but incorrect characters and hallucinates nonexistent headers.

A book that cannot be ingested leaves a marker naming the reason, so the shortfall can be checked instead of assumed. There are 118 such markers, of which 110 fall inside the taught curriculum and account exactly for the gap between the 263 books taught and the 153 ingested, and the remaining 8 are Urdu or dual-medium editions outside it. Table VI groups them.

Two of the 118 are ours, and we would have reported the corpus shortfall as wholly a property of the source material had the markers not recorded a reason. One book exhausted the disk mid-ingestion. The other produced a vector containing `NaN`, which `pgvector` refused, and because a book commits in one transaction the whole book rolled back. Both causes

TABLE VI
EVERY BOOK THE PIPELINE COULD NOT INGEST, BY RECORDED REASON. *ORIGIN* DISTINGUISHES A PROPERTY OF THE PUBLISHED MATERIAL FROM A PROPERTY OF THE MACHINE WE RAN ON.

Recorded reason	Books	Origin
No archive published (HTTP 404)	88	source
Legacy font: no usable chapters	27	source
Corrupt archive (`Bad CRC-32`)	1	source
Disk exhausted mid-ingestion	1	**ours**
`NaN` in an embedding vector	1	**ours**
Total	**118**	

are transient and neither is a statement about the book, yet `record_dead_end()` wrote the same permanent marker it writes for a corrupt archive. The two are recoverable by deleting their markers. One of them is the class 11 Physics text.

The `NaN` is the more interesting of the two. A single chunk embedding to `NaN` is enough to lose an entire textbook, and nothing upstream checks for it. The vector is written straight to a column that happens to reject it. Had the column been permissive the corpus would have gained a passage at infinite distance from every query, which fails quietly instead of loudly. We report it as a defect found by audit, not by test, because that is what it was.

V. Two Decisions the Results Rest On

Two implementation choices sit underneath every retrieval number in this paper, and both were arrived at by watching a simpler version fail.

Passages are 1,200-character windows with 200 characters of overlap, but the window is not taken at the character count. Each window is trimmed back to the last paragraph break within its final quarter, or failing that the last sentence end. A chunk cut mid-sentence embeds poorly and, because retrieved passages are shown to the student as the source of the answer, also reads as broken. The trim bounds how much text a boundary adjustment can discard while ensuring a chunk almost always ends where a human would end it.

The platform must decide which class's material answers a question, because the same topic at class 6 and class 12 requires different explanations. The decision is made from the retrieved passages themselves. Each class is scored as **the sum of its three strongest passages**, and the highest-scoring class wins. The rule is narrow, and it was reached by discarding the two obvious alternatives, each of which fails in a way we observed.

Summing every passage lets a class win on volume. A chemistry question matched eight weak passages in class 1 picture books and three strong ones in the class 10 chapter that actually teaches it. Class 1 won. The corpus composition that makes this a real risk and not a hypothetical one is given in Section VI.

Taking the single best passage fails in the opposite direction, and so does averaging. A class with one passage at 0.66 beats a class with three at 0.60–0.62 because its mean is that lone passage. One high match can be a passing mention that happens to share wording. Three consistent ones mean the topic is taught there.

Capping the sum at three passages bounds what volume can contribute while still rewarding corroboration. The rule is not principled in any deeper sense. It is the smallest rule that resists both observed failures, and it is reported here so that a reader can see the shape of the problem and not only the parameter.

VI. Cross-Lingual Retrieval

Every retrieval and grounding result in this section and the next was measured against a single corpus state, recorded here so that the numbers are interpretable and reproducible. The database held **153 textbooks**, 138 English and 15 Hindi, comprising 1,542 chapters and **42,995 indexed passages**, with an embedding present for every passage (42,995 of 42,995; a passage lacking one is invisible to retrieval and is not otherwise detectable). All twelve classes are represented, ranging from 3 books at class 1 to 34 at class 12.

This is 153 of the 263-book taught curriculum. The remaining 110 are not pending but unobtainable. NCERT publishes no archive for most of them, one class 5 archive is corrupt, and the rest are the legacy-font Hindi readers of Section IV, which the pipeline rejects instead of storing as gibberish. Each is recorded with its reason, so the shortfall can be audited rather than asserted. Auditing it found two books whose loss was not the source material's. One run exhausted the disk, and one chunk embedded to a non-finite vector, which a single-transaction commit turned into a whole rolled-back book. Both have since been recovered and the corpus now holds 155 books and 43,621 passages. Every result in this paper is measured against the 153-book state recorded here and has not been re-measured against the larger one.

Composition is load-bearing for one result, not incidental to it. English science and mathematics books outnumber the Hindi readers roughly nine to one, which is what makes the grade-attribution result of Section V a test and not a formality. A retriever that defaults to the majority of its corpus would answer every Hindi-literature question from a science chapter.

Absolute cosine similarities should be read as characteristic of this corpus and not as a property of the method. Recall in particular is not estimated. There is no ground-truth relevant set for these books, and no such claim is made.

BGE-M3's cross-lingual capability does not extend to romanized Hindi, the predominant input mode for Indian students using standard keyboards. The same chemistry question was embedded in three scripts (Table VII).

TABLE VII
SCRIPT IMPACT ON RETRIEVAL QUALITY

Script	Top Retrieval	Correct
English	Class 10 at 0.607	Yes
Devanagari	Class 10 at 0.564	Yes
Romanized Hindi	Class 1 at 0.520	**No**

Romanized Hindi drifts to class 1 picture books matching on surface-level token overlap. This failure provides a natural ablation study comparing naive direct retrieval (MultiRAG)

against a translation-then-retrieval (tRAG) approach. Direct retrieval across scripts failed entirely. By rewriting every question into a short English search query before embedding (following Ma et al. [4]), the vector search is successfully forced back into the correct semantic neighborhood.

Retrieval was evaluated over **22 queries: two topics in each of the eleven supported Indian languages**, against the English-medium portion of the corpus. Holding the topic constant across languages makes language the only variable that moves. One topic uses ordinary vocabulary (the focusing of the human eye). The other is *photosynthesis*, whose name in most of these languages is a compound of morphemes meaning "light" and "joining". The queries are committed with the benchmark, not only their results.

Five configurations were measured (Fig. 2), from a lexical baseline to the deployed system. A query counts as correct only when the *top* passage is on topic.

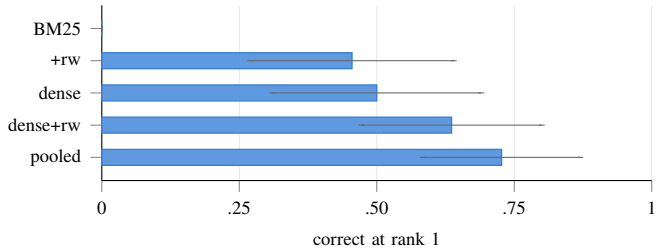

Fig. 2. The five retrieval configurations over 22 queries, with 95% Wilson score intervals [25]: BM25 0/22, with rewriting 10/22, dense 11/22, dense with rewriting 14/22, and the deployed pooling of both 16/22. The intervals overlap across the three dense configurations, which is why this paper claims an ordering and not a margin. The lexical baseline's interval excludes all three, and no sample-size argument softens that.

The lexical baseline retrieves nothing at all. BM25 [24] returns zero hits for all 22 queries, not merely poor ones. A Devanagari, Tamil or Perso-Arabic query shares no tokens with an English corpus, so the inverted index has nothing to match. Lexical retrieval is not weaker at this task, it is structurally incapable of it, and that is the argument for carrying a 568 M-parameter embedding model on a 4 GB machine. Given an English query from the rewrite it becomes viable at once (45%), which locates the difficulty precisely. The barrier is the script, not the vocabulary.

Multilingual RAG has been studied as a setting in its own right [9], including the tendency of such pipelines to answer in the language of the retrieved context instead of the question [10], which the separation of query language from answer language here is designed to prevent. The two arms fail on different inputs, and that is the result. Embedding the question directly handles ordinary vocabulary well (9 of 11) and collapses on compound terms (2 of 11), because the embedding attends to the constituent morphemes and retrieves optics for a word that means "light-joining". Rewriting inverts both. It normalises the compound to its English canonical form and recovers 8 of 11, while losing three ordinary-vocabulary cases whose original phrasing had been more precise than its English paraphrase.

Neither is therefore the right configuration on its own. Pooling both retrievals scores **16 of 22**, above either arm, because the union recovers what each misses instead of averaging them. Fig. 3 gives the deployed configuration language by language, as the mean reciprocal rank of its two queries.

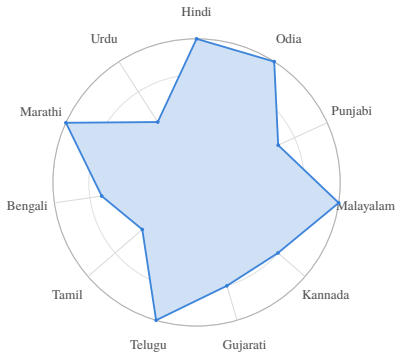

Fig. 3. Mean reciprocal rank of the deployed configuration for each language, averaged over the two topics. Rings mark 0.25, 0.5, 0.75 and 1.0; a language reaches the outer ring when both of its queries put an on-topic passage first. The two indentations that reach halfway are Urdu and Tamil, each of which answers one topic at rank 1 and returns nothing on photosynthesis.

The totals conceal which languages fail and how badly. Table VIII gives the deployed configuration per language and topic, with the rank of the first on-topic passage where it is not first. A third run of the benchmark, reported here, reproduced the totals of Fig. 2 exactly.

TABLE VIII
POOLED RETRIEVAL PER LANGUAGE. *1* MEANS AN ON-TOPIC PASSAGE RANKED FIRST; *rk* MEANS IT RANKED *k*TH; — MEANS NONE APPEARED IN THE TOP FIVE. BM25 RETURNED NO HITS FOR EVERY CELL IN THIS TABLE.

Language	Eye	Photo.	Language	Eye	Photo.
Hindi	1	1	Kannada	r2	1
Urdu	1	—	Malayalam	1	1
Marathi	1	1	Punjabi	1	r4
Bengali	r3	1	Odia	1	1
Tamil	1	—			
Telugu	1	1			
Gujarati	r2	1			

Most of the shortfall is strictness, not absence. Six queries are not correct under the rank-1 criterion, but four of them place an on-topic passage at rank 2 to 4. Counting a correct passage anywhere in the top five, pooled retrieval succeeds on **20 of 22**. Only Urdu and Tamil photosynthesis return nothing relevant at all. The strict criterion is the right one to report, because a student reads the first passage, but the distinction matters for what to fix. A system that has already retrieved the right passage and ordered it second needs reranking, not better embeddings. The cross-encoder reranker named in Table II is exactly that component, and it is integrated but not wired into the retrieval path. It is worth saying why not, because the obvious reason turns out to be wrong: peak resident set with the embedder and the reranker both loaded and twelve candidates scored is 2,083 MB, against 2,094 MB for the embedder alone, so the 4 GB budget is not what excludes it, which makes these four queries the strongest available argument for wiring it in, and the reason we report the component as present instead of omitting it.

The two genuine failures share a cause. Urdu and Tamil photosynthesis are the only queries where nothing on topic appears. Both are compound or borrowed terms whose morphology gives the embedding nothing to attach to. The Urdu *ziyai talif* is Perso-Arabic in origin and shares no root with either the English word or the Sanskrit-derived compounds the other nine languages use, and the Tamil rewrite returned a paraphrase that lost the term instead of normalising it. The failure is lexical, not script-level, which is consistent with the BM25 result, where supplying an English rewrite lifted a method that had returned nothing at all to 45%.

What 22 queries can and cannot establish. The intervals are wide and overlap for the three dense configurations. Pooling scores two queries above rewriting alone, at most four discordant pairs, which no test resolves at this n. We therefore claim only that pooling was never worse and that the ordering held across runs. One comparison is unambiguous. The lexical baseline's $[0.00, 0.15]$ excludes every dense interval, and no sample-size argument rescues a method returning nothing on all 22 queries. Separating the narrower differences needs a query set in the hundreds, which takes native speakers of eleven languages, not compute.

The rewrite is a language-model call and is not deterministic at temperature zero. Repeated runs of this benchmark returned 16 and 17 of 22, so the pooled figure should be read as 16 ± 1. The ordering of the five configurations was stable across runs, which is the claim the table is making. This is an empirical justification for a design that had been adopted on reasoning alone, and it also explains the earlier failure mode in which a mistranslated rewrite ("Rahim" rendered as "Rahman") silently replaced a question that would have retrieved correctly.

Two queries fail in every configuration, both on photosynthesis. The Urdu term (*ziyai talif*, Perso-Arabic in origin) shares no morphology with either the English word or the Sanskrit-derived compounds the other nine languages use, so neither arm has a handle on it. The Tamil failure has a different cause: the rewrite returned a paraphrase that dropped the term instead of normalising it, and direct embedding placed the passage fifth.

VII. Memory Optimization

BGE-M3's weights were evaluated at both full (float32) and half (float16) precision (Table IX).

TABLE IX
Precision Comparison for BGE-M3

Metric	float32	float16
Weight memory	2,166 MB	1,083 MB
Encode 4 queries	2,255 ms	1,197 ms
Model load time	8.5 s	8.1 s
Cosine fidelity (mean)	—	0.999998
Cosine fidelity (min)	—	0.999964

Half precision halves the weights and encodes $\sim 1.88\times$ faster. Fidelity, measured over 400 corpus passages rather than a handful of queries, is 0.999998 on average and 0.999964 at worst. Whether that matters is a question about rankings, so it was tested as one: over eight queries the top twelve passages are the same twelve under both precisions, in the same order for six of the eight, with the first difference at rank nine. Context is assembled from six passages, so nothing reported here is affected, but half precision can reorder a list rather than cannot.

Measurement against arithmetic. An earlier analysis summed component sizes to ~ 1.6 GB and concluded comfortable fit within 4 GB. A dedicated benchmarking script measures actual peak RSS through `getrusage(RUSAGE_SELF)`, a high-water mark surviving page eviction, unlike `ps` which reported 46 MB for a process holding a 1 GB model (Table X).

TABLE X
Serving Pipeline Memory Footprint (Measured)

Pipeline Stage	Peak RSS
Interpreter start	14 MB
Configuration imported	265 MB
BGE-M3 loaded (fp16)	534 MB
Query encoded	**2,094 MB**
pgvector search complete	2,094 MB

The actual peak is **2,094 MB**, not the 1,600 MB predicted (Fig. 4). Loading model weights accounts for only 534 MB (`safetensors` memory-maps them). The first forward pass adds 1,560 MB as weight pages become resident and attention workspaces are allocated. The system fits within 4 GB with ~ 1.4 GB usable headroom (accounting for ~ 500 MB of operating system overhead), not the 2.4 GB arithmetic implied.

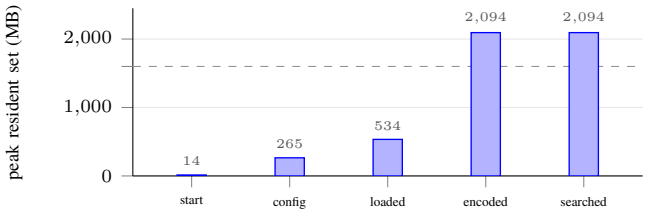

Fig. 4. Peak resident set through the serving pipeline, from interpreter start through configuration import, model load, the first query encoding, and a completed vector search. The dashed line is the 1,600 MB an earlier analysis predicted by summing component sizes. Loading the weights accounts for 534 MB because `safetensors` memory-maps them; the 1,560 MB that follows arrives on the first forward pass, when those pages become resident and the attention workspaces are allocated.

A single process ingesting 263 books reached book 8 and entered an unrecoverable state: swap at 11.4 GB of 12.2 GB maximum, the process in uninterruptible I/O wait. `SIGKILL` is not delivered in that state. The solution is batch processing: one process per six books, with ~ 20 seconds of model reloading per batch.

VIII. Reading Support for Brahmic Script Readers

Dyslexia support tooling is overwhelmingly designed for English. Most Indian students read a Brahmic script where the fundamental unit is the **akshara**, a consonant cluster carrying an inherent or explicit vowel. The segmentation algorithm recognizes the virama (halant) in each Brahmic script as the signal that two consonants are conjoined, operating generically

across Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Tamil, Telugu, Kannada, and Malayalam.

Two details: (1) sentence counting must honor the danda, not only the Latin period; (2) Flesch-Kincaid is computed for English only, since akshara counts are not syllable counts.

The first simplification attempt produced text *harder* than the original (Table XI).

TABLE XI
READABILITY MEASUREMENT: FLESCH-KINCAID GRADE LEVEL

Version	FK Grade
NCERT source text	7.0
First simplification attempt	**8.2** (harder)
Measure-and-keep approach	**6.9** (easier)

The corrected approach measures the original, rewrites with readability as sole objective, and retains whichever version is easier.

Letter and word spacing (on by default) is the only intervention with strong evidence [13]. Dyslexia-specific fonts are offered as preferences but labeled as weakly evidenced [14], [15]. Nothing in the module diagnoses any condition. It adjusts presentation.

IX. SECURITY ANALYSIS

A comprehensive audit identified twelve vulnerabilities, all repaired (Table XII).

TABLE XII
SECURITY AUDIT FINDINGS (SELECTED)

#	Vulnerability / Consequence
1	Safety pipeline import error caught by bare `except`: every response marked safe without evaluation
2	JWT `type` claim unchecked: 7-day refresh tokens accepted as 30-minute access tokens
3	`validate_required()` never called: production booted with no JWT secret
5	Embedding column as `double precision[]`: no vector index possible
7	RLS compared `NULL=NULL`: shared curriculum invisible to all users
9	Sentry received student PII despite `send_default_pii=False`

Vulnerability #1 is the most consequential. The safety pipeline was designed as a three-pass content filter, but an import path error caused the entire pipeline to fail to load. The error was caught by a bare `except` clause returning `(response, True)`, making the content safety system structurally incapable of blocking any response since deployment.

A policy that hid the curriculum from everybody (#7). Multi-tenancy placed row-level security on the content table with a single policy:

```
USING (organization_id =
current_setting('app.current_org_id')::uuid)
```

That is correct for a school's own uploads and wrong for NCERT textbooks, which belong to no school and must be readable by every student. A shared row has `organization_id IS NULL`, so the policy evaluates `NULL = NULL`, which is `NULL`, which PostgreSQL treats as false. Shared content was therefore not merely unowned but invisible to every user, permanently, and ingesting the curriculum at all was impossible. The repair adds a second policy under which a `NULL` organization means shared. Policies for the same command are OR'd, so tenant isolation is untouched and a row owned by one school remains visible only to that school. Writes were deliberately not opened. The existing policy has no `WITH CHECK`, so PostgreSQL applies its `USING` clause as the insert check and a tenant still cannot promote its own content to global. Only the table owner, which is what migrations and the ingestion job connect as, can publish shared curriculum.

This defect is characteristic of the class the audit kept finding. It is not a missing check but a check that is present, plausible, and evaluates to the wrong thing on a value nobody tested with. It produced no error, no log line and no failed request. It produced an empty result set, which is indistinguishable from a corpus that has not been ingested yet.

A token type nobody checked (#2). The JWT implementation validated signature and expiry but not the `type` claim, so a refresh token, issued with a seven-day lifetime for the sole purpose of obtaining new access tokens, was accepted anywhere an access token was expected. The practical effect is that the thirty-minute access-token lifetime, and every security argument resting on it, was fourteen times longer than stated for any client holding a refresh token. The repair checks the claim and the tests now assert the rejection, because a test that only exercises the valid path cannot fail when a validation step is removed.

X. GROUNDING AND DRIFT DETECTION

A retrieval-augmented system can retrieve the correct chapter and still generate a fabricated answer, which is the failure mode surveyed across generation tasks by Ji et al. [28] and the one retrieval is supposed to remove. When asked about a specific story from the Hindi curriculum, the system retrieved the correct chapter, then described events that never occurred. The class was correct, the chapter was correct, the language was correct, and every detail was invented.

The **grounding score** computes cosine similarity between the generated answer and the source passages. Reference-free RAG evaluation frameworks score a comparable property, faithfulness, with a language model as judge [26]; the metric used here is a single cosine similarity, which costs one encoder pass and no second model. Lexical overlap cannot serve the purpose because the platform produces answers in a different language from the source passages. BGE-M3's cross-lingual embedding property makes the metric functional in exactly this case.

Calibrating on a distribution. The threshold was originally set at 0.55 from four hand-verified answers, three correct at 0.583–0.701 and one fabricated at 0.489. Four points that happen to separate are not a calibration, and the difficulty is obtaining ungrounded answers in quantity. A model instructed to hallucinate does not fail the way a grounded pipeline fails.

The negatives are therefore constructed, not solicited. Every answer is generated normally from its own retrieved passages, then scored twice: against the passages it was written from (*matched*), and against another question's passages (*mismatched*). A mismatched pair is precisely the failure the metric exists to catch (fluent, well-formed, on a different subject) and it requires no human labelling and no second generation pass. Sixteen questions spanning subjects and classes yield 16 matched and 240 mismatched pairs.

Grounding need not be measured against the concatenation of the retrieved passages, so four estimators were compared over identical generations (Fig. 5). The two runs of the experiment, with independently generated answers, agreed to within 0.007 on every entry.

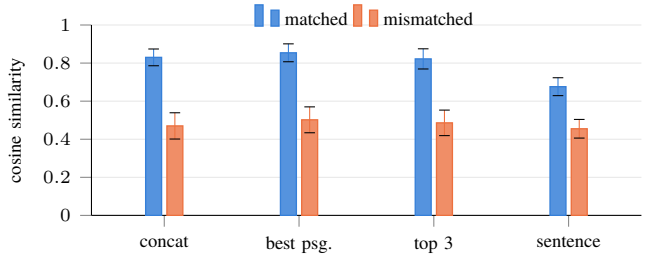

Fig. 5. Four grounding estimators over 16 matched and 240 mismatched pairs, with one standard deviation. What separates the populations is the distance between the two bars, not the height of either.

The simplest estimator is also the best. Scoring the answer against all retrieved passages joined separates the two populations by 0.360, and no refinement improves on it. Scoring against the single best passage raises both means without widening the gap, and scoring sentence by sentence narrows it, because a correct answer contains connective and pedagogical sentences that appear in no passage, and averaging over them dilutes the signal the metric depends on.

The boundary was in the wrong place. The distribution shows that 0.55 separates the populations but sits too low. Thirty-five of the 240 mismatched pairs, one drifted answer in seven, score above it and pass as grounded. At 0.715 none do, while no matched answer falls below it. Balanced accuracy rises from 0.927 to 1.000, and the number of correct answers needlessly regenerated stays at zero. The deployed threshold was raised accordingly. The original four points were not wrong, merely too few to locate the boundary.

That 1.000 should be read for what it measures. The negatives are answers written from *another question's* passages, so the task the score is perfect at is separating an answer from a different subject: topical drift, which is the failure we observed in deployment. It is not a measurement of hallucination detection in general, and an answer that stays on its retrieved topic while inventing a detail inside it would score as grounded here. The threshold is calibrated for the failure it was built to catch, and we claim no more than that.

The embedder is not judging its own geometry. BGE-M3 retrieved these passages and then scored the answers against them, so the separation could be a property of one embedding space and not of the text. Re-scoring the same generations settles it. Character 3–5 gram TF-IDF, not neural at all, separates the populations by 0.359, and a distilled multilingual MiniLM by 0.471, against BGE-M3's 0.360 (balanced accuracy 1.000, 0.996, 1.000). The gap is a property of the answers. Only the threshold is particular to the embedder deployed.

XI. Consolidated Results

The platform is validated by **490 automated tests**, of which **457 pass and 33 are skipped, with zero failures and zero errors.** The skips are declared, not silent. They are tests requiring a loaded model or a service this environment does not run, and the counts here are read from the suite's own JUnit report, not from a summary line.

Table XIII summarizes key metrics.

TABLE XIII
Consolidated System Metrics

Metric	Value
NCERT catalog coverage	559 textbooks
Taught curriculum	263 textbooks
Ingested corpus (all results)	153 textbooks
Supported languages	11
API routes	72
Ingestion throughput	140 s/book (1.38×)
Ingestion time, 153 books	∼6.0 hours
Peak serving memory (fp16)	2,094 MB
fp16 cosine fidelity (mean)	0.999998
Memory reduction (fp16)	50%
Encoding speedup (fp16)	1.88×
Response latency (warm)	∼3 s
Grounding threshold (calibrated)	0.715
Grounding balanced accuracy	1.000
Security vulns fixed	12
Automated tests	457 pass, 33 skip, 0 fail

XII. The Rewrite as a Failure Surface

The query rewrite is the one component that improves retrieval and degrades it in the same breath, and the ablation of Section VI exists because reasoning alone could not say which effect dominated.

Rewriting normalises a compound term to its English canonical form, which is why it lifts photosynthesis retrieval from 2 of 11 to 8 of 11. It also paraphrases, and a paraphrase can be less precise than the original. Three ordinary-vocabulary queries that direct embedding answers correctly are lost when rewritten. The two arms fail on disjoint inputs, so neither dominates and the union outperforms both. This is a property of these failure modes, not a general claim about rewriting.

Three consequences follow from the rewrite being a generation and not a transformation, and all three are visible in the measurements.

It is *not deterministic* at temperature zero. Repeated runs of the same benchmark returned 16 and 17 of 22, which is why the pooled figure is reported as 16 ± 1 and why the claim made from the table is the ordering, not the margin.

It can *silently substitute*. An early failure replaced "Rahim" with "Rahman" in a rewritten question about a Hindi poet, and the pipeline then retrieved correctly for a question the student had not asked. Nothing in the retrieval path can detect this. The retrieval succeeded, the grounding score was high,

and the answer was coherent. The defect is upstream of every check the system has.

It is *hosted*. The rewrite leaves the machine, so a system whose corpus and index are local still transmits the question. The paper states this in the abstract because the alternative, describing the platform as offline on the strength of local retrieval, is the claim this project has already had to retract once.

Pooling both retrievals is not an ensemble in any interesting sense. It is a union of two candidate sets before ranking, costing one extra vector search of approximately 20 ms. Choosing between the arms per query would require knowing in advance whether the query contains a compound term, which is the classification problem the rewrite was introduced to avoid. The union is the configuration that requires no such knowledge, and the measurement is what justifies preferring it.

XIII. Deployment and Portability

The 4 GB target is a claim about the machines Indian schools own, and those are predominantly Windows desktops. A memory measurement taken on Apple hardware is evidence about the software's footprint and not about performance on the target, and the gap between the two is where portability defects live.

The dependency manifest carried no environment markers, so an installation on any platform fetched `mlx`, `mlx-lm` and `coremltools`. None of the three publishes a wheel outside macOS and the first two are Apple Silicon only, so installation on Windows aborted before any other dependency was fetched. The code was already prepared for their absence. Every import of them sits inside a function behind `try`/`except`, and device selection already falls back through CUDA to CPU, so the platform the paper targets was excluded by the manifest alone. The markers now confine them to Apple Silicon, and an unused `python-magic` dependency was removed instead of marked. Nothing imports it, and on Windows it installs cleanly and then fails at runtime looking for a library pip does not supply.

Everything the platform needs to know about its host (installed memory, available memory, swap, peak resident set) is read through a single abstraction over `psutil`, which reports identically on Linux, Windows and macOS. That abstraction replaced direct system calls answering only on macOS: `sysctl hw.memsize`, `memory_pressure`, `sysctl vm.swapusage` and `getrusage`. One call site sat outside an exception handler and therefore raised instead of degrading, and the benchmark substantiating the 4 GB figure imported the POSIX-only `resource` module at module scope, making it unrunnable on precisely the platform whose claim it exists to support.

Peak resident set is the one quantity with no portable source. `getrusage` is POSIX-only. Windows exposes a peak through the process handle. The abstraction reads whichever exists and returns a flag when it has fallen back to current resident set, rather than presenting a smaller number as a high-water mark. A related unit defect is worth recording because it is silent: `ru_maxrss` is bytes on macOS and kilobytes on Linux, so identical code reports figures a factor of 1024 apart on two POSIX systems.

We have removed the known obstacles to installing and running on Windows and Linux, and we have not measured the result on either. No figure in this paper is offered for a platform we did not run on, and the portability work is reported as the removal of defects, not as evidence of performance.

XIV. Failures That Reported Success

Three defects in this project shared a property worth naming. Each produced a system that reported success while doing nothing. They are recorded because the measurement discipline in this paper exists in response to them, and because a reader deciding how much to trust the numbers is entitled to know what the numbers replaced.

Vector search had never run. The retrieval code queried with the pgvector cosine operator while `embeddings.embedding` was declared `double precision[]`, so every search failed on `operator does not exist: double precision[] <=> vector`. The migration that should have created the HNSW indexes instead skipped them, logging *"bypassing HNSW index creation due to missing pgvector extension"*, created a plain btree named `idx_embeddings_content_id_hnsw`, and then printed that it had built three HNSW indexes. The index name and the log message both described an object that did not exist.

Underneath was an environment fault, not a coding one. Homebrew builds pgvector only for the PostgreSQL versions it packages, so on the 14.19 server in use the extension was present for 17 and 18 and absent for 14. `CREATE EXTENSION` had been attempted, had failed, and the failure had been handled by continuing.

A book commits in a single transaction, so a failure rolls it back entirely and the book never appears in the database. The batched runner selects whatever is *not* in the database. Any book that fails deterministically is therefore selected again on every pass, forever. One overnight run spent **1,178 batches to ingest 16 books**: a class 5 archive with a `Bad CRC-32`, and legacy-font Hindi readers whose chapters are all correctly rejected, were chosen and re-chosen indefinitely. Books returning HTTP 404 already left a marker and were skipped. These two failure modes left none. The fix is the dead-end record of Section IV, and its own limitation, that it does not distinguish a transient cause from a permanent one, is the defect reported there.

The application role held privileges on none of the 42 tables. Reads returned empty instead of raising, so an empty corpus and an unreadable corpus were indistinguishable from the application's point of view, and the ingestion that appeared to succeed had written nothing the serving path could see.

XV. Threats to Validity

Construct. The grounding evaluation scores an answer against passages. A fluent answer reusing the vocabulary of its context scores well whether or not its claims are true. Section X shows the separation is not an artefact of one embedder, which is a different question from whether cosine

similarity measures factual grounding. It does not. It measures whether the answer is about the passages it was given.

Internal. Every retrieval and grounding number comes from one corpus state, one machine, and one hosted generation endpoint. The rewrite and the generation are model calls that are not deterministic at temperature zero, which is why the pooled retrieval figure is reported as 16 ± 1 and the grounding experiment was run twice with independently generated answers before either was reported.

External. The evaluation covers two topics per language and one curriculum. Nothing here establishes that the ordering of retrieval configurations transfers to another corpus, and the absolute cosine similarities certainly do not. The 4 GB claim is a measurement of peak resident set on one Apple M1. It is evidence about the software's footprint, not about performance on the Windows machines the target deployment assumes.

Absent. There is no human evaluation. No teacher or student has assessed whether an answer this system produces is pedagogically useful, and no measurement in this paper substitutes for that. A system that retrieves the right chapter, stays grounded in it, and fits in memory may still teach badly.

XVI. Reproducibility

Each result in this paper is produced by a committed script, named in Table XIV, run against a recorded corpus state. The corpus is 153 textbooks, 1,542 chapters and 42,995 passages with an embedding for every passage. The catalog, the ingestion pipeline and the dead-end markers reconstruct it from NCERT's published archives without redistributing them. The cross-lingual queries are committed in their original scripts, not transliterated, so a reader can inspect what was actually asked.

TABLE XIV
What produces each reported result.

Result	Script
Cross-lingual ablation	`cross_lingual_retrieval.py`
Grounding calibration	`grounding_estimators.py`
Estimator independence	`grounding_independent.py`
Encoder batch check	`grounding_batch_check.py`
Serving footprint	`serving_footprint.py`

The grounding scripts cache their generations, so a re-run rescores identical answers and separates a change in the metric from a change in what the model said. That distinction was not academic. An early run of the calibration reported balanced accuracy 0.451, and because it cached nothing its generations cannot be examined and it cannot be explained. Every figure reported here comes from a run whose inputs survive.

XVII. Discussion

The measurement-driven methodology reveals a recurring pattern. Intuitive engineering assumptions are frequently contradicted by empirical measurement, and the contradictions are more informative than the assumptions.

The legacy font detection illustrates this most starkly. A quality metric searching for damaged Devanagari ranked the most corrupted books as the best, because corruption manifested as the *absence* of Devanagari, which the metric interpreted as perfection. The memory estimation follows the same pattern. Summing static sizes to 1.6 GB misses the 2.1 GB transient peak. The grounding score addresses a fundamental RAG limitation. A system can retrieve correctly, generate fluently, pass every surface check, and still fabricate content.

Availability. The platform, the ingestion pipeline and every benchmark reported here, including the 22 cross-lingual queries in their original scripts and the grounding calibration, are available at https://github.com/rachitranka25/SHIKSHASETU. The corpus itself is not redistributed. The pipeline reconstructs it from NCERT's own published archives.

Rights and ethics. NCERT publishes these textbooks for free public download. The system stores only locally derived text and vectors, redistributed with neither the code nor this paper, and a deployment beyond research use should confirm the applicable terms instead of inferring them from availability. No human subjects took part and no student data was collected. Every measurement here is taken from the system itself.

Limitations. (1) The grounding negatives are constructed by mismatching answers to other questions' passages, not collected from observed hallucinations. This captures topical drift, which is the failure we have seen, but not an answer that stays on topic and invents a detail within it. (2) Cross-lingual retrieval is evaluated on two topics per language. Broader topic coverage would tighten the estimate. (3) 27 legacy-font Hindi books remain unprocessable. (4) Memory measurements have run-to-run variance of up to 414 MB.

XVIII. Conclusion and Future Work

This paper presented Shiksha Setu, a local-retrieval AI tutoring platform whose pipeline addresses the complete NCERT catalog of 559 textbooks and which delivers instruction in eleven Indian languages on consumer hardware with as little as 4 GB of RAM. The reported evaluation rests on 153 ingested textbooks and 42,995 indexed passages spanning all twelve classes. Every performance claim has been substantiated through kernel-level measurement, and engineering failures have been documented alongside their corrections.

The principal contributions are: (1) corpus construction with legacy-font corruption detection affecting 66% of Hindi textbooks, (2) cross-lingual retrieval measured over 22 queries in eleven languages, with an ablation showing that pooling the raw and rewritten queries retrieves correctly for 16 ± 1 of 22 against 11 and 14 for either alone, and that a lexical baseline retrieves none, (3) a grounding metric separating grounded from drifted answers by 0.360 in cosine similarity over 256 pairs, at a calibrated threshold that admits none of the 240 negatives where the previously shipped value admitted 35, (4) half-precision inference halving memory at a mean cosine fidelity of 0.999998, (5) akshara-level readability for nine Brahmic scripts, and (6) a security audit repairing twelve production vulnerabilities.

Future work targets: conducting a human-in-the-loop pilot study with actual educators to evaluate pedagogical utility, expanding the hallucination annotation dataset (100+ response

pairs) for robust grounding calibration, integrating a cross-encoder reranker to resolve compound-term failures (e.g., photosynthesis), implementing Entity Linking for polysemous term disambiguation, and developing a Walkman-Chanakya-to-Unicode transliterator.

References

[1] D. Kakwani *et al.*, "IndicNLPSuite: corpora, benchmarks and pre-trained models for Indian languages," in *Findings of EMNLP*, 2020.

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[3] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.

[4] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting for retrieval-augmented large language models," in *Proc. EMNLP*, 2023.

[5] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[6] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: multi-lingual, multi-granularity text embeddings via self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[7] A. Gala *et al.*, "IndicTrans2: accessible machine translation for all 22 scheduled Indian languages," *Trans. Mach. Learn. Res. (TMLR)*, 2023.

[8] S. Khanuja *et al.*, "MuRIL: Multilingual representations for Indian languages," 2021.

[9] Z. Xu *et al.*, "Multilingual retrieval-augmented generation for knowledge-intensive task," *arXiv preprint arXiv:2504.03616*, 2025.

[10] Y. Chen *et al.*, "Language drift in multilingual RAG: characterization and decoding-time mitigation," *arXiv preprint arXiv:2511.09984*, 2025.

[11] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using HNSW graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019, pp. 3982–3992.

[13] M. Zorzi *et al.*, "Extra-large letter spacing improves reading in dyslexia," *Proc. Nat. Acad. Sci. (PNAS)*, vol. 109, no. 28, pp. 11455–11459, 2012.

[14] L. Rello and R. Baeza-Yates, "Good fonts for dyslexia," in *Proc. ACM SIGACCESS Conf. Comput. Accessibility (ASSETS)*, 2013.

[15] S. M. Kuster, M. van Weerdenburg, M. Gompel, and A. Bosman, "Dyslexie font does not benefit reading in children with or without dyslexia," *Annals of Dyslexia*, vol. 68, pp. 25–42, 2018.

[16] S. Nag, "Early reading in Kannada: The pace of acquisition of orthographic knowledge and phonemic awareness," *J. Res. Reading*, vol. 30, no. 1, pp. 7–22, 2007.

[17] S. Nag and M. J. Snowling, "Reading in an alphasyllabary: Implications for a language-universal theory of learning to read," *Sci. Stud. Reading*, vol. 16, no. 5, pp. 404–423, 2012.

[18] J. P. Kincaid, R. P. Fishburne, R. L. Rogers, and B. S. Chissom, "Derivation of new readability formulas for Navy enlisted personnel," Naval Tech. Training Command, Rep. 8-75, 1975.

[19] W. Xu, C. Callison-Burch, and C. Napoles, "Problems in current text simplification research: New data can help," *Trans. Assoc. Comput. Linguist. (TACL)*, vol. 3, pp. 283–297, 2015.

[20] P. Micikevicius *et al.*, "Mixed precision training," in *Proc. ICLR*, 2018.

[21] Khan Academy, "Khanmigo: an AI tutor for learners and assistant for teachers," 2023. [Online]. Available: khanmigo.ai

[22] Ministry of Electronics and Information Technology, Government of India, "BHASHINI: National Language Translation Mission," 2022. [Online]. Available: bhashini.gov.in

[23] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023.

[24] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[25] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[26] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. EACL: System Demonstrations*, 2024, pp. 150–158.

[27] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[28] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, art. 248, 2023.

[29] Dept. of School Education and Literacy, "UDISE+ 2024–25 report," Ministry of Education, India, 2025.

[30] Registrar General of India, "Census of India 2011, Table C-17: bilingualism and trilingualism," 2011.